

# Chapter 2

Variables, expressions and statements

# Outline

- Overview
  - What is Program?
  - What is Python?
  - How to use python?
- Variables,
- Expressions and Statements
- Primitive data types and values
- Data Types & Casting
- Debugging
  - Types of errors
- Comments

# Overview: What is a program?

- A program is a sequence of instructions that specifies how to perform a computation.
- The computation might be something mathematical, such as solving a system of equations or finding the roots of a polynomial,
- but it can also be a symbolic computation, such as searching and replacing text in a document or
- compiling a program.
- The details look different in different languages, but a few basic instructions appear in just about every language

# Overview: What is Python?

- **Python** is a very popular general-purpose interpreted, interactive, object-oriented, and high-level programming language.
  - Also supports functional and structured programming methods
- Python is dynamically-typed and garbage-collected programming language.
- It was created by [Guido van Rossum](#) during 1985- 1990
- *Python design philosophy emphasizes code readability*
  - *Achieve with the use of significant indentation*
- *And Simplicity, making it easy to learn*
- It is Open Source which means its available free of cost.

# Overview: What is Python cont'd

- it is versatile and can be used to create many different things.
- It's widely used in web development, data science, artificial intelligence, scientific computing, and many other fields.
- Python code is easy to read and write syntax makes it an excellent choice for beginners.
- For example, here's how you print "Hello, World!" in Python

```
print("Hello World!")
```

# Overview: What is Python cont'd

- **Python is interpreted language:** meaning, you can run your code as soon as you write it without the need to compile it first.
  - This makes the development process faster and easier.
- **Python is dynamically typed:** meaning, you don't need to declare the type of a variable when you create one.
  - Python figures it out on its own:

```
x = 10    # No need to specify that x is an integer  
y = "Hello" # No need to specify that y is a string
```

# Overview: What is Python cont'd

- **Python comes with rich standard library:**
  - that includes modules and packages for everything from file I/O to web development and beyond.
  - Meaning, you often don't have to write everything from scratch.
- **Python has massive and active community**
  - There are countless tutorials, forums, and resources available for learning and troubleshooting.

# Overview: How to use python?

- Using Python involves several steps, from installation to writing and running your first script.
- Here's a comprehensive guide to get you started

## 2. Install Python

- **On Windows:**

- Download Python:** Go to the [official Python website](https://www.python.org/) and download the latest version for Windows.
- Run the Installer:** Double-click the downloaded installer.
- Add Python to PATH:** During installation, check the box that says "Add Python to PATH" and then click "Install Now."

- **On macOS:**

- Download Python:** Go to the [official Python website](https://www.python.org/) and download the latest version for macOS.
- Run the Installer:** Open the downloaded '.pkg' file and follow the instructions to install Python

- **On Linux:**

- Python usually comes pre-installed on most Linux distributions. You can check the installed version by running:

**python3 --version**

If not installed, you can install it using your package manager. For example, on Ubuntu

**sudo apt update**

**sudo apt install python3**



# Overview: How to use python cont'd

## 2. Write Your First Python Script

- You can write Python code using any text editor, however, using an Integrated Development Environment (IDE) can make the process easier.
- Some popular options are VSCode, PyCharm, and Jupyter Notebook.
- **Using a Text Editor or IDE:**
  - **Open Your Text Editor:** Open your preferred text editor
  - **Write Code:** Write the following Python code to print “Hello World!” and save your script as ‘hello.py’

```
print("Hello, World!")
```

# Overview: How to use python cont'd

## 3. Edit and Run Your Python Script

- You can run your python script using a command line, an IDE or a python interactive shell

### a) Using the Command Line:

1. **Open Command Line:** Open Command Prompt (Windows), Terminal (macOS), or your terminal emulator (Linux).
2. **Navigate to Your Script:** Use the 'cd' command to navigate to the directory where your script is saved.
3. **Run the Script:** Run your script by typing:

`python hello.py`

Or, if you're using Python 3.x, you might need to type:

`python3 hello.py`

# Overview: How to use python cont'd

## 3. Edit and Run Your Python Script cont'd

### b) Using an IDE

- Many IDEs allow you to run your Python scripts directly within the editor. For example, in PyCharm:
  1. **Open PyCharm:** Open your project or create a new one.
  2. **Create a New Python File:** Right-click on your project directory, select '**New**' then '**Python File**', and name it '**hello.py**'
  3. **Write your code in the python code editor** and save it
  4. **Run the Script:** Click the '**Run**' button or right-click the code and select '**Run**' '**hello.py**'

# Overview: How to use python cont'd

## 3. Edit and Run Your Python Script cont'd

### c) Using a Python Interactive Shell

- You can also use the Python interactive shell to write and run Python code line-by-line:
  1. **Open Command Line:** Open Command Prompt (Windows), Terminal (macOS), or your terminal emulator (Linux).
  2. **Start Python Shell:** Type '**python3**' and press enter
  3. **Write Python Code:** Write Python code directly in the interactive shell. For example

```
>>> print("Hello, World!")
```

4. And press enter

# Overview: How to use python cont'd

## 4. Install Packages with pip

- Python has a rich ecosystem of libraries and packages that you can use to extend its functionality.
- You can install packages using '**pip**' which is the package installer for python
- **Example: Installing a Package**

1. **Open Command Line:** Open Command Prompt (Windows), Terminal (macOS), or your terminal emulator (Linux).
2. **Install a Package:** Use 'pip' to install a package. For example, to install the '**requests**' library:

```
pip install requests
```

Or, if you're using Python 3.x, you might need to type:

```
pip3 install requests
```

## 5. Explore Python Documentation and Tutorials

- The [official Python documentation](#) is a great resource for learning about different Python modules and functionalities.
- There are also many tutorials available online that can help you learn Python step-by-step.

# Variables and data types

- A **variable** is a name that refers to a value which is used to refer to the value later

```
student_age = 20  
message = 'Hello'
```

```
Unit_price = 20  
quantity = 14  
unit_price = 15
```

- You can change the contents of a **variable** in a later statement
- More formally a **variable** is a named place in the memory where a programmer can store data and later retrieve the data using the **variable name**
  - Programmers get to choose the names of the **variables**
  - The **variables can get their value via the assignment operator '='**
    - It is used to change the value of the variable

# Variables cont'd

- The right hand side of the assignment operator can be any valid Python expression.
- This includes a wide range of elements, from simple literals to complex expressions and function calls.
- examples of what can be on the right-hand side of an assignment operator:

## 1. Literals

### a) Numeric literals:

```
myVariable = 100
```

### b) String literals:

```
myCourse = 'Programming'
```

### c) Boolean literals:

```
classBegins = True
```

```
classEnds = False
```

## 2. Variables:

```
num1 = 5
```

```
num2 = num1 # num2 is assigned the value of num1
```

# Variables cont'd

- The right hand side of the assignment operator can be

## 3. Expressions

### a) Arithmetic expressions

totalPrice = unitPrice \* quantity #assume unitPrice and Quantity are initialized

### a) Logical expressions

is\_valid = (username == "admin") and (password == "secret")

### b) Boolean expressions

is\_greater = x > y

is\_equal = a == b

### c) Function call

sqrt\_num = sqrt(4)

### a) Data Structures

I. **List:** numbers = [1, 2, 3, 4, 5]

II. **Tuples:** coordinate = (10,20)

III. **Dictionaries:** student = {"name" : "Selam" , "GPA" : 3.9}

IV. **Sets:** fibnoccii\_lessThan8 = {1 ,1, 2, 3, 5}



# Takeaway exercise

1. What would be printed if each of the following snippet is executed?

```
num1 = 5
num2 = num1 * 6
result = 'Your result is: '
print(result, num2)
```

```
num1 = 8
num2 = num1
num1 = 3
print(num2, num1)
```

```
print(isAuthorized)
unitPrice = 10
quantity = 5
totalPrice = unitPrice * quantity
print("Total price is ", totalPrice)
```

```
username = 'userD'
password = 'mySecNum'
isAuthorized = username == 'userID' and password == 'mySecNum'
print(isAuthorized)
```

# Variables and Data Types

- Variables are named entities/ placeholder used to store data.
  - Python is dynamically typed
  - Python is case sensitive # age = 19 and Age = 19 are different
- Python supports several data types including integers, floats, strings, and Boolean

```
name = "Selam" # string
age = 19      #integer
height1 = 1.71 #floating point
is_student = True    #Boolean
```

# Variables and data types

A valid variable name is

A combination of alphabets, numbers and the underscore (\_) that doesn't start with numbers nor it contains other characters (space?)

Should not be any of the reserved words

if = 10 # invalid variable name

for = 100 # invalid variable name

Alternative:

if\_value = 10 # valid

for\_count = 100 # valid

```
# valid variable names
name = "Selam"
_age = 19
height1 = 1.71
is_student = True
```

```
#invalid variable names
2nd_name = "Dinu" # Starts with a digit
my-name = "Selam" # Contains a hyphen
class = "Programming" # Uses a reserved word
```

# Named constants

- They are similar to variables: a memory location that's been given a name.
- Unlike variables their contents *shouldn't* change.

```
>>> PI = 3.14
```

- In python, it is the responsibility of the programmer to keep constants from changing
  - Their all caps naming convention is an assistance to remind the programmer

# Literals

Literal/unnamed constant/magic number: not given a name, the value that you see is literally the value that you have.

```
>>> netPay = 300000 – (300000 * 0.35)
```

# Reserved words

- The built-in module 'keyword' enables you to check for reserved keywords programmatically.

```
import keyword
# List all reserved keywords
print(keyword.kwlist)
```

- Reserved words printed:

['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']

# Expressions and Statements

- An **expression** is a combination of values, variables, operators, and function calls that is evaluated to produce another value.
- Examples:
  - Literal values: '33', 'hello'
  - Variables: 'num', 'myVariable'
  - Operations: '3+2', 'num1-num2', 'var1 \*var2',
  - Function calls: 'len("Hello")', 'max(10,2)'

# Expressions and Statements

- **Expression** example

```
# Arithmetic expressions
result = 3 + 4 # result is 7
# String expressions
greeting = "Hello" + " " + "World" # greeting is "Hello World"
# Function call expressions
length = len("Python") # length is 6
# Boolean expressions
is_greater = 5 > 3 # is_greater is True
```

- Takeaway question: identify the operations in each expression above



# Statements

- A **statement** is an instruction that the Python interpreter can execute.
- Statements do something—like
  - creating variables,
  - displaying values, or
  - controlling the flow of execution in a program.
- Examples of Statements:
  - Assignment statements: `num1 = 22`
  - Control flow statements: `'if', 'for', 'while', 'try', etc`
  - Function definitions: `'def myFunction():'`
  - Class definitions: `'class MyClass:'`
  - Import statements: `import math`

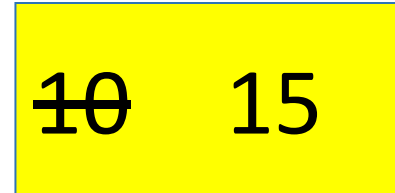
# Assignment statement

- An **assignment statement** assigns a value to a variable

num1 = 10

num1

num1 = num1 + 5



~~10~~ 15

- Basic form:

<variable> = <expr>

# Assignment Statements

- We assign a value to a variable using the **assignment** statement (=)
- An **assignment statement** consists of an **expression on the right hand side** and a **variable** to store the result

`num1 = 10`

`num1 = 5 * num *(num1-5)`

- The right hand side expression enclosed in rectangle along with the assignment operator makes an assignment statement that updates the value of the variable num1 with result of evaluation of the expression

# Mnemonic Variable Names

- Since programmers have a choice in how to choose variable names, there is a bit of “best practice”
  - name variables to help you remember what you intend to store in them  
(“mnemonic” = “memory aid”)
- Which of these namings are helping you remember them?

```
x112 = 35.0  
x358 = 12.50  
x1321 = x112 * x358  
print (x1321)
```

```
num1 = 35.0  
num2 = 12.50  
num3 = num1 * num2  
print (num3)
```

# Numeric Expressions and operator precedence

- Numeric expressions in Python are combinations of numbers, variables, and operators that evaluate to a number.
- Python evaluates expressions based on the precedence of the operators.
- the precedence from highest to lowest:
  1. **Parentheses** □ `'()'`
  2. **Exponentiation** □ `'**'`
  3. **Unary operation** □ `'+', '-'`
  4. **Multiplication, division, reminder** □ `'*', '/', '//', '%'`
  5. **Addition, subtraction** □ `'+', '-'`

# Operator Precedence Rules

- **Associativity**

- Most operators in Python are **left-associative**, meaning they are evaluated from left to right.
- Exponentiation `**` is **right-associative**, meaning it is evaluated from right to left.
- Find the result

```
result = 3 + 2 * 5  
print(result)
```

```
result = (3 + 2) * 5  
print(result)
```

```
result = 2 ** 3 ** 2  
print(result) # Output: 512
```

```
result = 2 ** 3 ** 2  
print(result) # what would happen if you think it should be left associative?
```

# Types matters

- In Python variables, literals, and constants have a “**type**”
- Python knows the **difference** between an integer number and a string
- For example “+” means “addition” if something is a number and “concatenate” if something is a string

```
>>> num1 = 1 + 4
>>> print(num1)
5
>>> message = 'hello ' + 'there'
>>> print(message)
hello there
```

# Several **Types** of Numbers

- Numbers have two main types
  - Integers are whole numbers: -14, -2, 0, 1, 100,
  - Floating Point Numbers have decimal parts:  
-2.5 , 0.0, 98.6, 14.0
- There are other number types - they are variations on float and integer

```
>>> num = 1
>>> type(num)
<class 'int'>
>>> temp = 98.6
>>> type(temp)
<class 'float'>
```



# Mixing Integer and Floating

- When you perform an operation where one operand is an integer and the other operand is a floating point the result is a floating point
- The integer is converted to a floating point before the operation

```
>>> print(99 // 100)
0
>>> print(99 / 100.0)
0.99
>>> print(99.0 // 100)
0.0
>>> print(1 + 2 * 3 // 4.0 - 5)
-3.0
>>>
```

# Type Matters

- Python knows what “type” everything is (dynamically typed)
- Type implies permissible ops
- Some operations are prohibited
  - You cannot “add 1” to a string
- We can ask Python what type something is by using the `type()` function.

```
>>> message = 'hello ' + 'there'
>>> message = message + 1
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: cannot concatenate 'str'
and 'int' objects
```

```
>>> type(message)
<type 'str'>
>>> type('hello')
<type 'str'>
```

# String

- a string is a sequence of characters enclosed within either single quotes (' ') or double quotes (" ").
- It is an immutable data type, which means once a string is created, it cannot be modified.
- However, it is possible to create a new string by concatenating two or more strings

```
str1 = 'hello'  
str2 = 'world'  
str3 = str1 + str2  
print(str3)# result?
```

```
str4 = '33'  
str5= '1'  
print(str4+str5)#result?
```

# Boolean (bool) data type

- the Boolean data type is used to represent truth values.
- There are only two Boolean values: True or False
  - #note the case
- These values are used in, comparisons, logical operations, and conditional statements to control the flow of a program
- Comparison operators are used to compare values and they return a Boolean value: '==', '>', '<', '!=', '>=', '<='

```
>>> is_present = True
>>> type(is_present)
<class 'bool'>
```

```
>>> took_exam = False
>>> type(took_exam)
<class 'bool'>
```

The output `<class 'bool'>` indicates the variable is a boolean data type

# comparison operators in Boolean expressions

- What is the output of the following Boolean expressions?

```
>>> num1 = 10
>>> num2 = 20
>>> print(num1 == num2)
>>> print(num1 != num2)
>>> print(num1 > num2)
>>> print(num1 < num2)
>>> print(num1 >= 10)
>>> print(num2 <= 20)
```

# Logical operators in Boolean expression

- Logical operators are used to combine multiple Boolean expressions.
  - **and**: Returns **True** if both expressions are **True**.
  - **or**: Returns **True** if at least one expression is **True**.
  - **not**: Returns **True** if the expression is **False**, and vice versa
- The built-in function `bool()` converts a value to a Boolean type

```
print(bool(0))    # False
print(bool(1))    # True
print(bool(""))   # False
print(bool("Hi")) # True
```

# Logical operators in Boolean expression

- Some values are considered **True** and others are considered **False** when evaluated in a Boolean context
- Values considered '**True**' in Boolean context:
  - Non-zero numbers (both integers and floats)
  - Non-empty strings
  - Non-empty sequences (like lists, tuples)
  - Non-empty mappings (like dictionaries)
- Values considered '**False**' in Boolean context:
  - 'None'
  - 'False'
  - 0 (zero of any numeric type)
  - Empty sequences ('', '()', '[]')
  - Empty mappings '{}'

```
# Example 1#output?
num1 = 5
num2 = 10
# Check if both conditions are true
if num1 > 0 and num2 > 0:
    print("Both num1 and num2 are positive numbers")
else:
    print("At least one of the numbers is not positive")
# Example 2 #output?
var1 = -5
var2 = 10
# Check if at least one condition is true
if var1 > 0 or var2 > 0:
    print("At least one of var1 or var2 is a positive number")
else:
    print("Neither var1 nor var2 is a positive number")
if var1 and var2:
    print("Both are non-zero ")
```



# Type Conversions (Type casting)

- When you put an integer and floating point in an expression the integer is **implicitly** converted to a float
- You can control this with the built in functions `int()` and `float()`

```
>>> i_num = 42
>>> type(i_num)
<class 'int'>
>>> f_num = float(i_num)
>>> print(f_num)
42.0
```

# String Conversions

- You can also use `int()` and `float()` to convert between strings and integers
- You will get an **error** if the string does not contain numeric characters and casted to numeric type first

```
>>> str_val = '123'
>>> type(str_val)
<class 'str'>
>>> print(str_val + 1) #type mismatch
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: cannot concatenate 'str' and 'int'
>>> int_val = int(str_val) #cast to int
>>> type(int_val) #check its type
<class 'int'>
>>> print(int_val + 1) #addition is permitted 124
```

# Takeaway exercise

1. What would be printed? Check it on your IDE

```
# data type
name = "Selam" # string
age = 19      #integer
height = 1.71 #floating point
is_student = True    #Boolean
print(type(name))
print(type(age))
print(type(height))
print(type(is_student))
```

# Takeaway exercise

1. We can also get input from the user using “**input**”. What is wrong with the following code

```
num1 = input("enter first number: ") # assume you entered 9
num2 = input("enter second number: ")#assume you entered 2
num3 = num1/num2
Num4 = num1 //num2
print(num3, num4)
```

2. What is the output of the following?

```
num1 = input("enter first number: ") # assume you entered 9
num2 = input("enter second number: ")#assume you entered 2
num3 = int(num1)/int(num2)
num4 = int(num1)//int(num2)
print(num3, num4)
```

# Expressions and Statements

|              | Expression                                      | Statements                                                  |
|--------------|-------------------------------------------------|-------------------------------------------------------------|
| purpose      | Compute and return a value                      | Perform an action                                           |
| Usage        | Can be part of other expressions and statements | Stand alone and cannot be part of an expression             |
| Evaluates to | Always produce a value                          | Do not necessarily produce a value; they execute an action. |

```
# Assignment statement with an expression
y = 3 * (2 + 1) #exp `3 * (2 + 1)` is eval and assigned to `y`

# Print statement with an expression
print(2 + 2) # expression `2 + 2` is evaluated to 4, and printed
```

# Try it and see the output

```
# Assignment statement
num = 10
# Print statement
print(num)
# If statement
- if num > 5:
    print("x is greater than 5")
# For loop statement
- for i in range(3):
    print(i)
# Function definition statement
- def greet(name):
    print(f"Hello, {name}!")
greet('Students')
```

# Type casting

- There may be times when you want to specify a type on to a variable.
  - This can be done with casting.
  - Python is an object-orientated language, and as such it uses classes to define data types, including its primitive types.
- Casting in python is therefore done using constructor functions:
- `int()` - constructs an integer number from an integer literal, a float literal (by removing all decimals), or a string literal (if the string represents a number)
- `float()` - constructs a float number from an integer literal, a float literal or a string literal (if the string represents a number)
- `str()` - constructs a string from a wide variety of data types, including strings, integer literals and float literals
- `bool()` – constructs a Boolean type

# Type casting: try it to see the changes

```
x = int(1)      # x will be 1
y = int(2.8)    # y will be 2
z = int("3")    # z will be 3
```

```
x = float(1)    # x will be 1.0
y = float(2.8)  # y will be 2.8
z = float("3")  # z will be 3.0
w = float("4.2") # w will be 4.2
```

```
x = str("s1")   # x will be 's1'
y = str(2)      # y will be '2'
z = str(3.0)    # z will be '3.0'
```



# Boolean data type

```
x = 5
y = 10
# Equal to
print(x == y)    # Outputs: False
# Not equal to
print(x != y)    # Outputs: True
# Less than
print(x < y)     # Outputs: True
# Greater than or equal to
print(x >= y)    # Outputs: False
```

# Logical Operators

- Logical operators are used to combine Boolean expressions.
- They allow you to create more complex conditions by combining simpler ones.
- Here are the three main logical operators in Python: 'and', 'or', 'not'

```
x = 15
y = 20
z = 25
# Using 'and' operator
print(x < y and y < z)    # Outputs: True
# Using 'or' operator
print(x < y or y > z)    # Outputs: True
# Using 'not' operator
print(not (x == y))       # Outputs: True
```

# What is debugging?

- Programming is a complex process, and because it is done by human beings, it often leads to errors.
- programming errors are called **bugs** and the process of tracking them down and correcting them is called **debugging**.
- Three kinds of errors can occur in a program: **syntax errors, runtime errors, and semantic and/or logical errors**.
- It is useful to distinguish between them in order to track them down more quickly

# What is debugging: Syntax errors

- Syntax errors are the first errors you will make and the easiest to fix.
- It means that you have violated the “grammar” rules of Python.
- Python does its best to point right at the line and character where it noticed it was confused.
- The only tricky bit of syntax errors is that sometimes the mistake that needs fixing is actually earlier in the program than where Python noticed it was confused.
  - So the line and character that Python indicates in a syntax error may just be a starting point for your investigation.
  - Could occurs when there is missing colon or unmatched parentheses

# Example: missing colon

```
# Syntax error: missing colon after if statement  
if 10 > 5  
    print("10 is greater than 5")
```

```
#How to fix it? put colon after if statement  
if 10 > 5:  
    print("10 is greater than 5")
```

Takeaway exercise:

- Explore other examples of syntax errors in python

# What is debugging: Logic error

- A logic error occurs when
  - your program has good syntax but
  - there is a mistake in the order of the statements or
  - perhaps a mistake in how the statements relate to one another (the logic)
- the code runs but produces incorrect results due to a flaw in the logic

# Example: Logical error

```
# Logical error: incorrect calculation of average
numbers = [1, 2, 3, 4, 5]
average = sum(numbers) / len(numbers) + 1 # Incorrect logic
print(f"The average is: {average}")
```

```
#Fix it by correcting the logic
# Correct logic for calculating average
average = sum(numbers) / len(numbers)
print(f"The average is: {average}")
```

Takeaway exercise:

- Explore other examples of logical errors in python

# What is debugging: Semantic errors

- A semantic error occurs when
  - the code is syntactically correct and
  - runs without errors, but
  - the meaning or result of the code does not align with what the programmer intended.
- This is similar to a logical error but more about the intended **purpose** or **meaning**.
  - The code runs and is logically correct but does not do what the programmer intended (often due to misuse or misunderstanding of the intended logic).



# Example: Semantic error

- Imagine you want to calculate the square of the sum of two numbers. However, due to a semantic error, you accidentally calculate the sum of the squares of the two numbers instead.

```
# Semantic error: incorrect calculation
def square(x):
    return x * x

a = 3
b = 4
#Intended to calculate the square of the sum
#((a + b)^2)
# Incorrect logic: calculating a^2 + b^2
result = square(a) + square(b)
print(f"The result is: {result}")
```

```
# Correct calculation of the square of the sum
def square(x):
    return x * x

a = 3
b = 4

# Correct logic: calculating (a + b)^2

sum_ab = a + b
result = square(sum_ab)
print(f"The result is: {result}")
```

Takeaway exercise:

- Explore other examples of semantic errors in python

# What is debugging: runtime error

- occurs while the program is running.
  - This can be due to various reasons like
    - dividing by zero,
    - accessing an undefined variable,
    - out-of-bounds indexing, or
    - file not found

# Example: division by zero

```
# Runtime error: division by zero
numerator = 10
denominator = 0
# This will raise a ZeroDivisionError
result = numerator / denominator
print(f"The result is: {result}")
```

Takeaway exercise:

- Explore other examples of runtime errors in python

```
# Handling division by zero
numerator = 10
denominator = 0

if denominator != 0:
    result = numerator / denominator
    print(f"The result is: {result}")
else:
    print("Error: Cannot divide by zero")
```

# Takeaway exercise

- what is the response of each of the following Boolean expressions?

```
>>> 2 < 3
>>> 3 < 2
>>> 5 - 1 > 2 + 1
>>> 3 == 3
>>> 3 + 5 == 4 + 4
>>> 3 == 5 - 3
>>> 3 <= 4
>>> 3 >= 4
>>> 3 != 4
>>> 2 < 3 and 4 > 5
>>> 2 > 3 or False
```

# Takeaway Exercise

1. : Write a program that uses input to prompt a user for their name and then greets them.
2. Write a simple program that asks the user to enter two integers and displays their average
3. Write a program to prompt the user for hours and rate per hour to compute gross pay
4. Write a program which prompts the user for a Celsius temperature, convert the temperature to Fahrenheit, and print out the converted temperature
5. Write a program that asks a user to enter three test results, the first out of 20, the second out of 30 and the third out of 50. Compute the average score of the tests and print it to the screen

# Comments

- Comments in Python are used to explain code, making it easier to understand and maintain.
- They are ignored by the Python interpreter, so they do not affect the execution of the program.
- Python supports two types of comments:
  - single-line comments and
  - multi-line comments.
- **Single-line Comments**
  - Single-line comments begin with the hash character #
  - extend to the end of the line
  - are used for brief explanations or notes

# This is a single-line comment

num =15 # This comment explains that num is assigned the value 15

print(num) # Output will be 5

# Comments cont'd

## •multi-line comments

- Python does not have a specific syntax for multi-line comments like some other languages.
- Instead, multi-line comments can be created by using multiple single-line comments or by using multi-line strings (a pair of triple quote **"** or **"""**
  - While the latter method technically creates a string, if it's not assigned to a variable or used in any way, it effectively acts as a comment:

```
"""
```

This is a multi-line comment.

It spans multiple lines.

Useful for longer explanations.

```
"""
```

# Best practice of using comments

- **Keep Comments Clear and Concise:** Comments should be easy to read and understand.
- **Avoid Obvious Comments:** Do not write comments that are self-evident from the code
- **Use Comments to Explain Why, Not What:** Focus on explaining the purpose and rationale behind complex or non-obvious code, rather than describing what the code is doing.
- **Keep Comments Up-to-date:** Ensure comments are maintained and updated alongside the code they describe. Outdated comments can be misleading.
- **Comment Regularly but Judiciously:** Regularly use comments to explain complex logic, but avoid over-commenting. Strive for a balance.



# Example: multiple line comment

```
# Function to calculate the factorial of a number
def factorial(n):
    """
    This function returns the factorial of a given number n.
    Factorial of n (n!) is the product of all positive integers less than or equal to n.
    """
    if n == 0:
        return 1
    else:
        return n * factorial(n - 1)
# Calculate factorial of 5
result = factorial(5)
print(result) # Output: 120
```

# Takeaway exercise

- What is printed as a result of each of the following codes?

```
print(type('This is a string.'))  
print(type("And so is this."))  
print(type("""and this."""))  
print(type(''and even this...'))
```

# Triple quotes

- Strings enclosed with three occurrences of either quote symbol are called triple quoted strings.
  - They can contain either single or double quotes
- What will be printed as a result of each print statement below?

a)

```
print(''"Oh no", she exclaimed,"stop knocking at the door!"'"')
```

b)

```
message = """This message will  
    span several  
    ... lines."""  
print(message)
```

# End of Chapter 2